# Practical - 4

**Aim:**

To Implement and Time analysis of factorial program using iterative and recursive method

**Theory:**

Both iterative and recursive implementations of the factorial program share a linear time complexity of \(\mathcal{O}(n)\), but they differ drastically in their memory profiles. While the iterative approach uses constant space \(\mathcal{O}(1)\), the recursive approach consumes linear space \(\mathcal{O}(n)\) due to function call overhead

**Algorithm**

**Algorithm (Iterative Method)**

Start the program and read the value of n.

Initialize a variable fact = 1.

Repeat from 1 to n, multiplying fact by each number.

Display the factorial value.

Stop the program.

**Algorithm (Recursive Method)**

Start the program and read the value of n.

If n is 0 or 1, return 1.

Otherwise, return $n \times \text{factorial}(n - 1)$.

Display the factorial value returned by the recursive function.

Stop the program.

**Program Code**

```
#include <iostream>
#include <chrono>

// Function for Iterative Factorial
// Time Complexity: O(n)
// Space Complexity: O(1)
unsigned long long factorialIterative(int n) {
    unsigned long long result = 1;
    for (int i = 1; i <= n; ++i) {
        result *= i;
    }
    return result;
}

// Function for Recursive Factorial
// Time Complexity: O(n)
// Space Complexity: O(n) due to call stack
unsigned long long factorialRecursive(int n) {
    if (n <= 1) return 1;
    return n * factorialRecursive(n - 1);
}

int main() {
    int n;
    std::cout << "Enter a non-negative integer (e.g., 20): ";
    if (!(std::cin >> n) || n < 0) {
        std::cerr << "Invalid input! Please enter a non-negative integer." << std::endl;
        return 1;
    }

    // Measure Iterative Implementation
    auto startIter = std::chrono::high_resolution_clock::now();
    unsigned long long resIter = factorialIterative(n);
    auto endIter = std::chrono::high_resolution_clock::now();
```

```
    std::chrono::duration<double, std::nano> durationIter = endIter - startIter;

    // Measure Recursive Implementation
    auto startRec = std::chrono::high_resolution_clock::now();
    unsigned long long resRec = factorialRecursive(n);
    auto endRec = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::nano> durationRec = endRec - startRec;

    // Output Results
    std::cout << "\n--- Results for " << n << "! ---" << std::endl;
    std::cout << "Iterative Result : " << resIter << std::endl;
    std::cout << "Iterative Time   : " << durationIter.count() << " ns" << std::endl;
    std::cout << "------------------------------ " << std::endl;
    std::cout << "Recursive Result : " << resRec << std::endl;
    std::cout << "Recursive Time   : " << durationRec.count() << " ns" << std::endl;

  std::cout<<"Sangem Vijayendra
  Reddy\n";
  std::cout<<"92460118132\n";
  std::cout<<"5-EN18\n";

    return 0;
}
```

**Result :**

```
"C:\Users\Admin\Downloads\

Enter a non-negative integer (e.g., 20): 25

--- Results for 25! ---
Iterative Result : 7034535277573963776
Iterative Time   : 500 ns
-------------------------------
Recursive Result : 7034535277573963776
Recursive Time   : 500 ns
Sangem Vijayendra Reddy
92460118132
5-EN18

Process returned 0 (0x0)   execution time : 6.454 s
Press any key to continue.
```